ESP32 - Blinking LED

Project Overview

This project is to both learn how to use an ESP32 and explicitly document/apply the V-Model. Since this is my first project the objective is the standard “Hello World” of hardware, the blinking LED. There will be 3 stages; inputs will progress from timer → button → wifi, and the outputs will be an on-board LED as well as an external LED.

Requirements Traceability Matrix

ID	Requirement	Cycle	Design Ref	Code Module	Test Case(s)	Status
REQ-1	LED shall blink automatically every 200ms when idle	v1	Sys. Design 1.0	loop(), setup(), ledOn(), ledOff(), toggleLED()	TC-1.1 Unit (interval)	
REQ-2	Drive external LED within the safe limits of GPIOs (12mA recommended, 40mA max)	v1	Sys. Design 1.0		TC-1.2 Integration TC-1.3 System (50-cycle)	
REQ-3	Button press shall toggle LED	v2	Sys. Design 2.0	readButton()	TC-2.1 Unit (debounce) TC-2.2 Integration TC-2.3 System (Repeated Input)	
REQ-4	LED shall be toggle-able via Button press OR a WiFi HTTP request	v3	Sys Design 3.0	handleButton(), handleWifi(), WiFi setup	TC-3.1 Unit (endpoint) TC-3.2 Integration (HTTP→GPIO) TC-3.3 System (button regression)	

v1.0 - Timer

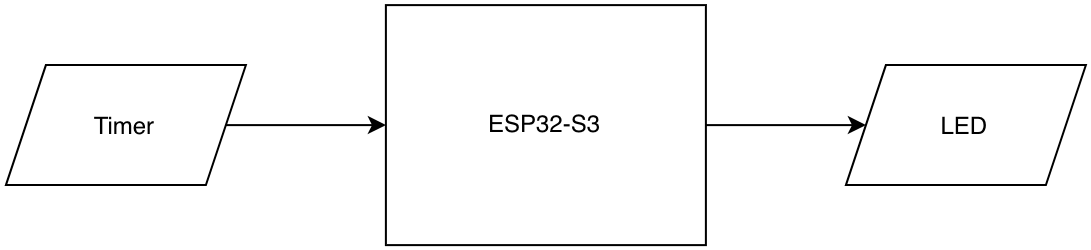
Requirement Analysis

- REQ-1: The system shall turn the LED on/off at a 200ms interval
- REQ-2: Drive external LED within the safe limits of GPIOs (12mA recommended, 40mA max)

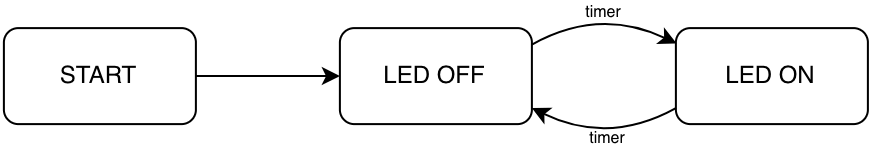
System Design

- GPIO48: On-board LED
- GPIO4: External LED

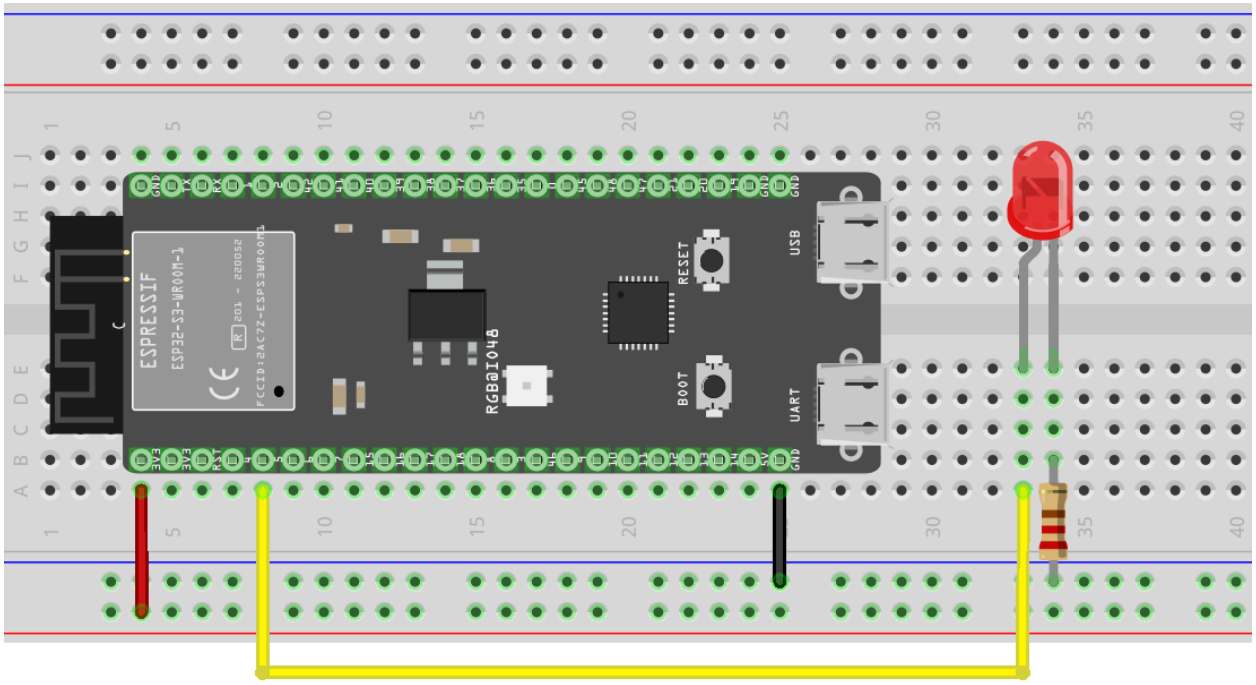
Block Diagram

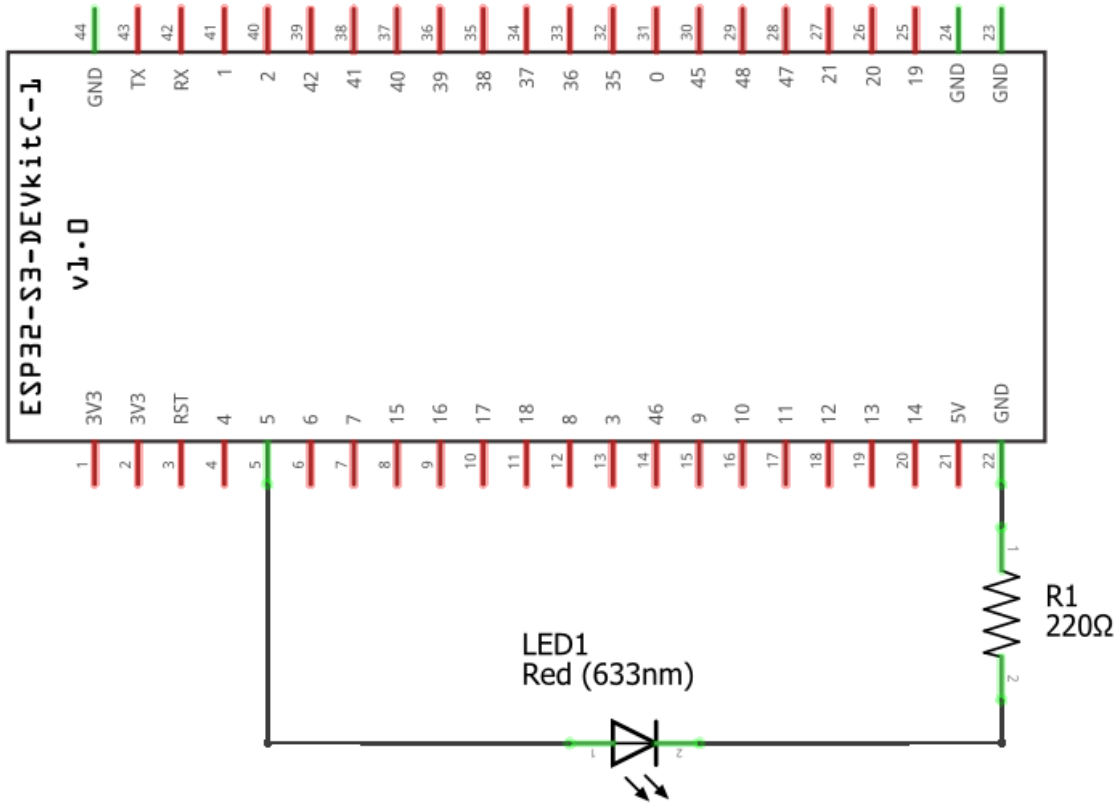


State Diagram



Architecture Design





Pin Map Table

PIN	Direction	Function	Connected To	Notes
GPIO48	Output	Drives on-board LED	On-board LED	
GPIO4	Output	Drives External LED	LED through resistor	

Module Design

setup() - configure pin modes, initialize serial bus (baud-rate)
loop() - call output functions with delay (timer)
ledOn() - turn LED ON
ledOff - turn LED OFF
toggleLED() - flip LED state

Unit Testing

- TC-1.1: Verify that the LED can be turned off, turned on, and toggled using serial monitor

Integration Testing

- Flash hardware
- TC-2.2: Measure current draw on GPIO4

System Testing

- Power cycle the board
- TC-2.3: Verify system is stable after 50+ cycles

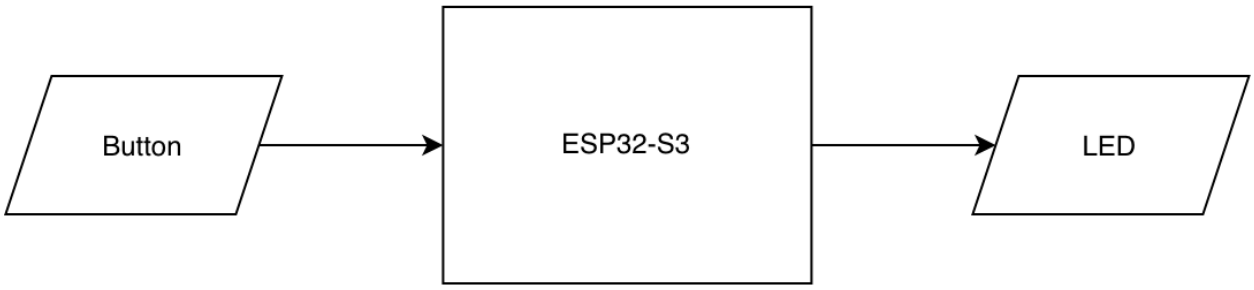
Acceptance Testing

- ☐ REQ-1: LED blinks at 200ms interval
- ☐ REQ-2: GPIO4 port current draw is within spec

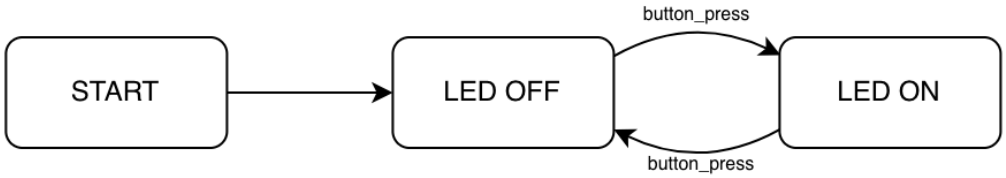
v2.0 - Button

- **Requirements Added:**
  - REQ-3: The system shall toggle the state of the LED in response to the button
- **System Design Delta:**
  - GPIO14: Push-Button

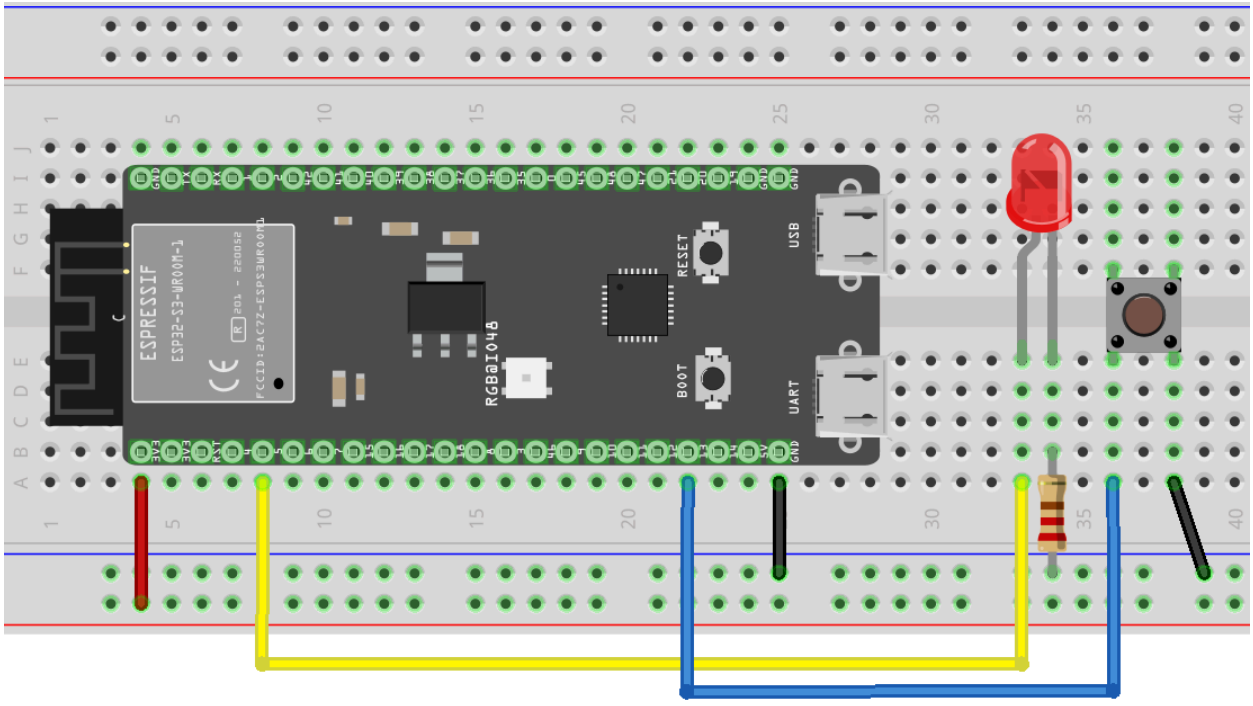
Block Diagram

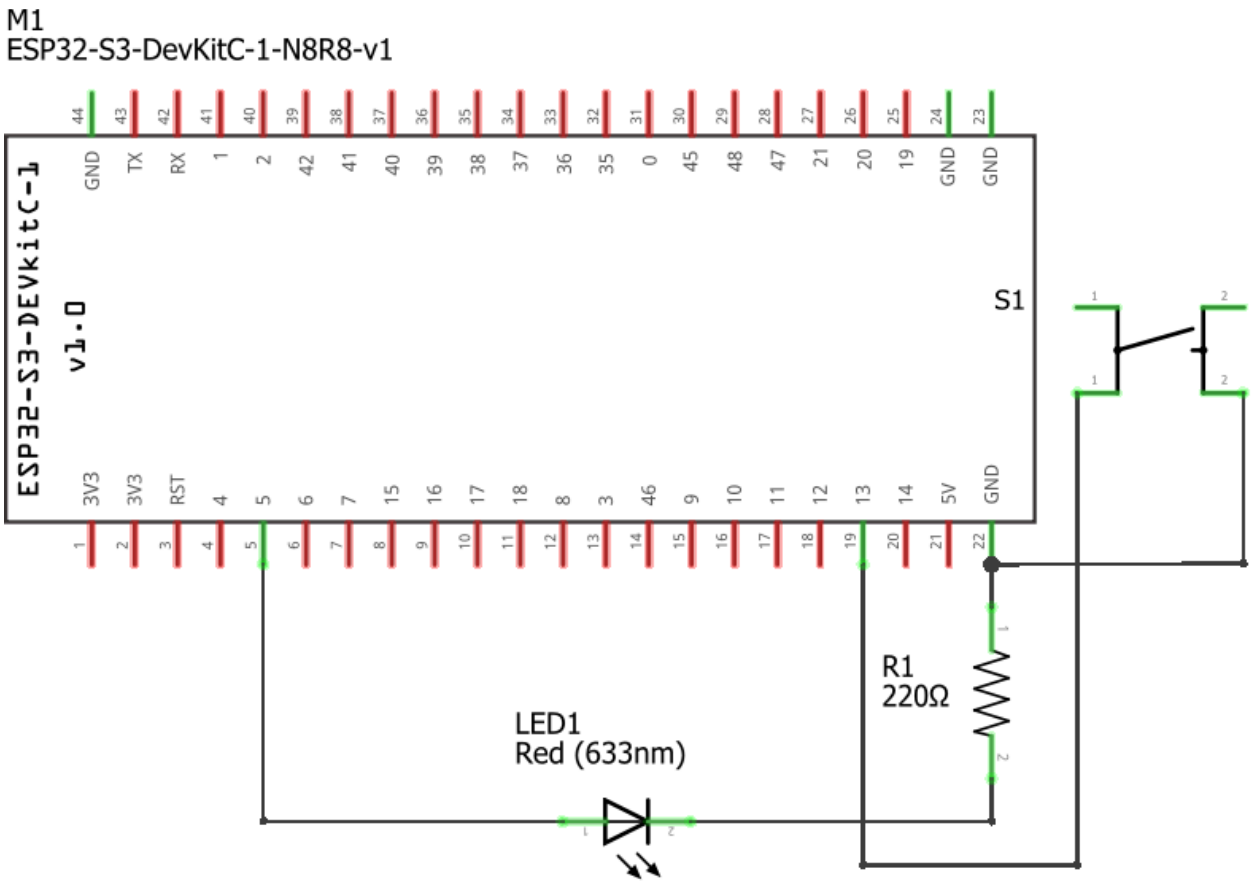


State Diagram



- **Architectural Delta:**





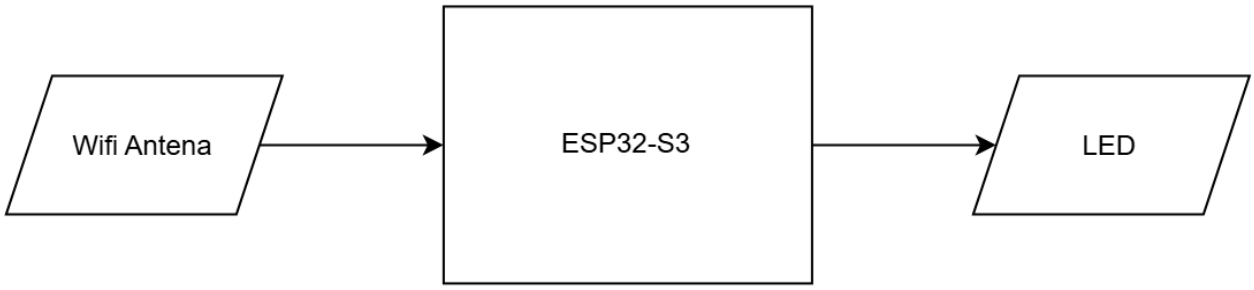
PIN	Direction	Function	Connected To	Notes
GPIO13	Input	Interface with Button	Push Button	Need to debounce, software: INTERNAL_PULLUP

- **Modules Added:**
  - **readButton()** - Debounce input
- **Unit Testing:**
  - TC-2.1: Verify readButton() reports a single single clean press with no false triggers (debounce)
- **Integration Testing:**
  - TC-2.2: Confirm a physical button press reliably toggles the real LED on the breadboard
- **System Testing**
  - TC-2.3: Repeat inputs to confirm repeated outputs
- **Acceptance Testing:**
  - ☐ REQ-3: Push button toggles the LED

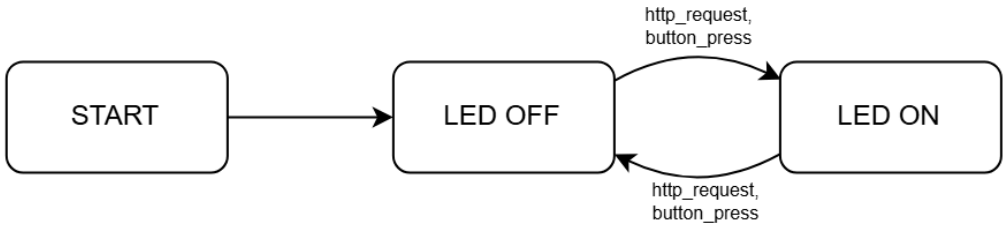
v3.0 - Wifi

- **Requirements Added:**
  - REQ-4: LED shall be toggle-able via Button press OR a WiFi HTTP request
- **System Design Delta:**
  - Wifi initialization and verification

Block Diagram



State Diagram



- **Modules Added:**
  - handleButton() - handler for button-interrupt
  - handleWifi() -handler for wifi request
  - wifiSetup() -wifi initialization to be called in setup()
- **Unit Testing:**
  - TC-3.1: Verify http endpoint requests are valid and return expected outputs
- **Integration Testing:**
  - TC-3.2: Confirm that calls to output functions interface with LED
- **System Testing**
  - TC-3.3: Test race conditions between button interrupt and wifi handling to ensure that neither request is ‘forgotten’
- **Acceptance Testing:**
  - ☐ REQ-4: Both wifi requests and button can control the LED with race conditions eliminated by interrupt handling